ETL Project — Master Consolidation Document

Project Title

Customer Churn ETL & Analytics Pipeline using Python, PySpark, SQL, and Power BI

---

1. Project Overview

1.1 Problem Statement

Telecom companies face major revenue loss because of customer churn. Large customer datasets contain patterns related to customer behavior, subscription usage, payment methods, contracts, and services. However:

- Raw data is unstructured and inconsistent
- Manual analysis is slow
- Data cleaning is repetitive
- Decision-makers need dashboards and insights
- Scalable processing is required for enterprise-level datasets

The project aims to build a complete ETL pipeline that:

1. Extracts telecom customer churn data
2. Cleans and transforms the data
3. Loads processed data into analytical storage
4. Generates insights and dashboards
5. Supports scalable processing using PySpark

---

2. Dataset Information

Dataset Used

IBM Telco Customer Churn Dataset

Source:

- Kaggle
- IBM Sample Dataset

Dataset Purpose

The dataset helps analyze:

- Why customers leave telecom services
- Which services influence churn
- Revenue-related behavior
- Contract-based retention
- Payment behavior

---

3. Technologies Used

Component	Technology
Programming Language	Python
Data Processing	Pandas
Big Data Processing	PySpark
Database	MySQL / PostgreSQL
Visualization	Power BI
Notebook Environment	Jupyter Notebook
IDE	VS Code
Version Control	Git + GitHub
Environment Management	Python Virtual Environment
ETL Orchestration (Optional Future Scope)	Apache Airflow
Data Storage	CSV / SQL Tables

---

4. Development Environment Setup

Folder Structure

```
ETL_project/
│
├── etl_env/
├── data/
│   ├── raw/
```

```
│   ├── processed/
│   └── backup/
│
├── notebooks/
│   ├── 01_data_understanding.ipynb
│   ├── 02_data_cleaning.ipynb
│   ├── 03_feature_engineering.ipynb
│   └── 04_visualization.ipynb
│
├── src/
│   ├── extract.py
│   ├── transform.py
│   ├── load.py
│   ├── utils.py
│   └── config.py
│
├── sql/
│   ├── schema.sql
│   └── queries.sql
│
├── dashboards/
│   └── churn_dashboard.pbix
│
├── reports/
│   └── project_documentation.docx
│
├── requirements.txt
├── README.md
└── main.py
```

---

5. Environment Setup Commands

Create Virtual Environment

```
python -m venv etl_env
```

Activate Environment

Windows

```
etl_env\Scripts\activate
```

Linux/Mac

```
source etl_env/bin/activate
```

---

6. Required Libraries

Install Dependencies

```
pip install pandas numpy matplotlib seaborn jupyter pyspark sqlalchemy mysql-
connector-python powerbiclient
```

requirements.txt

```
pandas
numpy
matplotlib
seaborn
jupyter
pyspark
sqlalchemy
mysql-connector-python
powerbiclient
```

---

7. ETL Workflow

```
Raw CSV Data
      ↓
Extract Phase
      ↓
Data Cleaning
```

```
      ↓
Transformation
      ↓
Feature Engineering
      ↓
Validation
      ↓
Load to SQL Database
      ↓
Power BI Dashboard
```

---

8. Detailed ETL Pipeline

8.1 Extract Phase

Objective

Read raw telecom churn data.

Code

```
import pandas as pd

file_path = '../data/raw/Telco-Customer-Churn.csv'

df = pd.read_csv(file_path)

print(df.head())
print(df.shape)
```

Key Tasks

- Read CSV file
- Validate columns
- Check dataset size
- Verify missing values

8.2 Understanding Data (EDA)

Initial Exploration

```
print(df.info())
print(df.describe())
print(df.columns)
```

Missing Values

```
print(df.isnull().sum())
```

Duplicate Records

```
print(df.duplicated().sum())
```

Churn Distribution

```
print(df['Churn'].value_counts())
```

8.3 Data Cleaning

Objective

Remove inconsistencies and prepare data for transformation.

Convert TotalCharges

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
```

Fill Missing Values

```
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)
```

Remove Duplicates

```
df.drop_duplicates(inplace=True)
```

8.4 Data Transformation

Convert Churn Column

```
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})
```

Encode Gender

```
df['gender'] = df['gender'].map({'Male': 1, 'Female': 0})
```

Create Tenure Group

```
bins = [0, 12, 24, 48, 72]
labels = ['0-1 Year', '1-2 Years', '2-4 Years', '4-6 Years']

df['tenure_group'] = pd.cut(df['tenure'], bins=bins, labels=labels)
```

8.5 Feature Engineering

Monthly Revenue Category

```
import numpy as np

conditions = [
    df['MonthlyCharges'] < 35,
    (df['MonthlyCharges'] >= 35) & (df['MonthlyCharges'] < 70),
    df['MonthlyCharges'] >= 70
]

choices = ['Low', 'Medium', 'High']
```

```
df['RevenueCategory'] = np.select(conditions, choices)
```

8.6 Data Validation

Validate Nulls

```
print(df.isnull().sum())
```

Validate Datatypes

```
print(df.dtypes)
```

8.7 Load Phase

Database Connection

```
from sqlalchemy import create_engine

username = 'root'
password = 'password'
host = 'localhost'
database = 'etl_churn_db'

engine = create_engine(
    f'mysql+mysqlconnector://{username}:{password}@{host}/{database}'
)
```

Load Data

```
df.to_sql('customer_churn', con=engine, if_exists='replace', index=False)

print('Data loaded successfully')
```

---

9. PySpark Implementation

Why PySpark?

PySpark is included to demonstrate:

- Distributed data processing
- Big data handling
- Enterprise scalability
- Parallel computation
- Industry-level ETL practices

This improves academic project value and practical exposure.

9.1 Start Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder
    .appName('CustomerChurnETL')
    .getOrCreate()
```

9.2 Load Dataset in PySpark

```
spark_df = spark.read.csv(
    '../data/raw/Telco-Customer-Churn.csv',
    header=True,
    inferSchema=True
)
```

9.3 View Schema

```
spark_df.printSchema()
```

9.4 Null Handling in PySpark

```
from pyspark.sql.functions import col

spark_df = spark_df.fillna({
    'TotalCharges': 0
})
```

9.5 Transformations in PySpark

```
from pyspark.sql.functions import when

spark_df = spark_df.withColumn(
    'ChurnFlag',
    when(col('Churn') == 'Yes', 1).otherwise(0)
)
```

9.6 Save Processed Data

```
spark_df.write.csv(
    '../data/processed/churn_output',
    header=True,
    mode='overwrite'
)
```

---

10. SQL Database Design

10.1 Database Schema

```
CREATE DATABASE etl_churn_db;
```

10.2 Table Creation

```sql
CREATE TABLE customer_churn (
    customerID VARCHAR(50),
    gender VARCHAR(10),
    SeniorCitizen INT,
    Partner VARCHAR(10),
    Dependents VARCHAR(10),
    tenure INT,
    PhoneService VARCHAR(10),
    MultipleLines VARCHAR(50),
    InternetService VARCHAR(50),
    OnlineSecurity VARCHAR(50),
    OnlineBackup VARCHAR(50),
    DeviceProtection VARCHAR(50),
    TechSupport VARCHAR(50),
    StreamingTV VARCHAR(50),
    StreamingMovies VARCHAR(50),
    Contract VARCHAR(50),
    PaperlessBilling VARCHAR(10),
    PaymentMethod VARCHAR(100),
    MonthlyCharges FLOAT,
    TotalCharges FLOAT,
    Churn VARCHAR(10)
);
```

---

11. Power BI Dashboard

Dashboard Objectives

Visualize:

- Total customers
- Churn percentage
- Revenue loss
- Contract-based churn
- Payment method analysis
- Monthly revenue trends

11.1 Suggested Dashboard Pages

Page 1 — Executive Summary

KPIs:

- Total Customers
- Churn Rate
- Total Revenue
- Active Customers

Page 2 — Customer Behavior

Charts:

- Churn by Contract
- Churn by Internet Service
- Churn by Gender

Page 3 — Revenue Analysis

Charts:

- Revenue by Tenure
- Monthly Charges Distribution
- High-risk Customer Segments

---

12. Suggested Visualizations in Python

Churn Count

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='Churn', data=df)
plt.title('Customer Churn Distribution')
plt.show()
```

Monthly Charges vs Churn

```
sns.boxplot(x='Churn', y='MonthlyCharges', data=df)
plt.title('Monthly Charges vs Churn')
plt.show()
```

---

13. Main Project Execution File

main.py

```
from src.extract import extract_data
from src.transform import transform_data
from src.load import load_data


def main():
    df = extract_data()
    transformed_df = transform_data(df)
    load_data(transformed_df)


if __name__ == '__main__':
    main()
```

---

14. Modular Python Files

14.1 extract.py

```
import pandas as pd


def extract_data():
    file_path = 'data/raw/Telco-Customer-Churn.csv'
    return pd.read_csv(file_path)
```

14.2 transform.py

```
import pandas as pd


def transform_data(df):
    df['TotalCharges'] = pd.to_numeric(
        df['TotalCharges'],
        errors='coerce'
    )

    df['TotalCharges'].fillna(
        df['TotalCharges'].median(),
        inplace=True
    )

    return df
```

14.3 load.py

```
from sqlalchemy import create_engine


def load_data(df):
    engine = create_engine(
        'mysql+mysqlconnector://root:password@localhost/etl_churn_db'
    )

    df.to_sql(
        'customer_churn',
        con=engine,
        if_exists='replace',
        index=False
    )
```

---

15. Jupyter Notebook Workflow

Notebook Sequence

Notebook	Purpose
01_data_understanding.ipynb	Initial EDA
02_data_cleaning.ipynb	Cleaning & preprocessing
03_feature_engineering.ipynb	Transformations
04_visualization.ipynb	Charts & insights

---

16. GitHub Workflow

Initialize Git

```
git init
```

Add Files

```
git add .
```

Commit

```
git commit -m "Initial ETL project setup"
```

Push to GitHub

```
git remote add origin <repository_url>
git push -u origin main
```

---

17. README Structure

README.md Contents

```
# Customer Churn ETL Project

## Objective
Build a complete ETL pipeline for telecom customer churn analysis.

## Technologies
- Python
- Pandas
- PySpark
- SQL
- Power BI

## Features
- Data Extraction
- Data Cleaning
- Transformation
- SQL Loading
- Dashboard Analytics

## Project Workflow
Raw Data → ETL → Database → Dashboard
```

---

18. Documentation Structure for Final Report

INDEX

Chapter	Topic
7	Database Design
8	ETL Workflow
9	Implementation
10	Testing
11	Dashboard & Outputs
12	Future Scope
13	Conclusion
14	References

---

19. System Design

19.1 Architecture Diagram (Logical)

```
CSV Dataset
     ↓
Python Extraction
     ↓
Cleaning & Transformation
     ↓
PySpark Processing
     ↓
SQL Database
     ↓
Power BI Dashboard
     ↓
Business Insights
```

19.2 Data Flow Diagram (DFD)

Level 0

```
User → ETL System → Database → Dashboard
```

Level 1

```
Raw Dataset
    ↓
Extract Module
    ↓
Transformation Module
    ↓
Validation Module
    ↓
Database Loader
    ↓
Visualization Layer
```

---

20. Testing

Types of Testing

Testing Type	Purpose
Unit Testing	Validate functions
Integration Testing	Validate ETL flow
Data Validation Testing	Validate transformed data
Dashboard Testing	Validate visual accuracy

---

21. Sample Test Cases

Test Case	Expected Result
CSV loads correctly	Dataset imported
Null handling works	No critical null values
SQL connection works	Data inserted successfully
Dashboard refresh works	Updated visuals shown

---

22. Future Scope

Possible Enhancements

- Apache Airflow integration
- Real-time streaming ETL
- Machine learning churn prediction
- AWS cloud deployment
- Snowflake data warehouse
- Kafka integration
- Automated alert systems

---

23. Academic Presentation Flow

Presentation Order

1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Overview
5. Technology Stack
6. ETL Workflow
7. Data Cleaning
8. PySpark Processing
9. Database Loading
10. Power BI Dashboard
11. Insights
12. Challenges
13. Future Scope
14. Conclusion

---

24. Important Insights You Can Mention

- Customers with month-to-month contracts churn more
- Electronic check payment users show higher churn
- High monthly charges correlate with churn
- Longer tenure customers are more loyal
- Fiber optic users may have higher churn percentages

---

25. Important Academic Justification for PySpark

Even though the dataset is manageable with Pandas, PySpark is implemented because:

- Industry ETL systems process massive datasets
- PySpark demonstrates scalability
- Distributed computing is part of modern data engineering
- It increases technical depth of the project
- It aligns with enterprise ETL architecture

---

26. Suggested Resume Description

Resume Project Entry

Customer Churn ETL & Analytics Pipeline

- Developed an end-to-end ETL pipeline using Python, Pandas, and PySpark
- Performed data extraction, transformation, cleaning, and SQL loading
- Built interactive Power BI dashboards for churn analysis
- Implemented scalable data processing concepts using PySpark
- Conducted exploratory data analysis and feature engineering

---

27. Key Learning Outcomes

- ETL pipeline architecture
- Data preprocessing
- SQL integration
- Big data fundamentals
- Dashboard development
- Data engineering workflow
- Analytical reporting
- Project structuring

---

28. Final Conclusion

The project successfully demonstrates a complete ETL workflow for telecom customer churn analysis using Python and PySpark. The pipeline extracts raw customer data, cleans and transforms it, loads it into a structured database, and visualizes insights through Power BI dashboards.

The implementation combines:

- Data engineering
- Data analytics
- Database integration
- Visualization
- Scalable processing concepts

This project provides both academic value and practical industry relevance.

---

29. Current Project Status (As Per Workflow)

Completed

- Environment setup
- Virtual environment configuration
- Library installation
- Project structure planning
- Initial ETL architecture
- Data understanding planning
- EDA planning
- Pandas-based workflow setup
- PySpark integration planning

Ongoing

- Data understanding notebook
- Exploratory Data Analysis
- Cleaning implementation
- Transformation logic

Pending

- SQL loading
- Power BI dashboard
- Documentation screenshots
- Final report formatting
- Testing documentation
- Presentation preparation

---

30. Recommended Next Steps

Immediate Next Actions

1. Import dataset into notebook
2. Perform detailed EDA
3. Document all findings
4. Start cleaning pipeline
5. Create transformation notebook
6. Build SQL schema
7. Create Power BI dashboard
8. Prepare screenshots
9. Generate final documentation
10. Push complete project to GitHub

---

31. Recommended Deliverables

Deliverable	Status
Source Code	Required
Jupyter Notebooks	Required
SQL Scripts	Required
Power BI Dashboard	Required
Project Documentation	Required
Presentation PPT	Required
GitHub Repository	Recommended

---

32. Suggested Screenshots for Documentation

Include Screenshots Of:

- VS Code environment
- Project folder structure
- Jupyter Notebook EDA
- Missing value handling
- PySpark schema
- SQL table
- Power BI dashboard

- Charts and insights
- Successful ETL execution

---

33. Suggested Viva Questions Preparation

Possible Questions

What is ETL?

Extract, Transform, Load process used for data integration.

Why PySpark?

To demonstrate scalable distributed data processing.

Why Pandas?

Efficient for exploratory analysis and medium-sized datasets.

Difference between Pandas and PySpark?

Pandas is single-machine processing. PySpark supports distributed computing.

Why Power BI?

Interactive business intelligence visualization.

Why use SQL database?

Structured storage and analytical querying.

---

34. Final Recommended Project Name Variants

1. Telecom Customer Churn ETL Pipeline
2. Customer Churn Analytics using ETL and PySpark
3. ETL-Based Telecom Customer Analytics System
4. Scalable Customer Churn Analysis Pipeline
5. Customer Retention Analytics using Python and Power BI

END OF MASTER CONSOLIDATION DOCUMENT